

LLM Inference Optimizer

A Plain-English Overview for Beginners

A complete, beginner-friendly guide to what this project is, the ideas behind it, what it achieved, and how. No prior machine-learning or GPU experience is assumed - every technical term is explained the first time it appears.

0. How to read this document

Sections 1-2 explain the problem and the vocabulary. Section 3 gives the big picture. Section 4 walks through the system one stage at a time. Section 5 shows the results in plain terms, Section 6 tells the story of the hard parts, and Sections 7-8 cover engineering practices and what comes next. If you only read two sections, read 3 (the big picture) and 5 (the results).

1. The problem, in everyday terms

A large language model (LLM) is the kind of AI behind chatbots like ChatGPT. Think of it as a brilliant but slow-to-respond expert: it knows an enormous amount, but to produce each word of an answer it must do a staggering amount of arithmetic. On a computer, that arithmetic runs on a GPU (a graphics chip that is very good at doing millions of small calculations at once).

Running a model to answer questions is called inference (as opposed to training, which is teaching the model in the first place). Inference is slow and expensive: it ties up costly GPUs and makes users wait. Anyone who runs these models cares about two questions:

- How do we make the model answer faster and serve more people at the same time?
- How do we prove it actually got faster - with real measurements, not guesses?

This project answers both: it speeds the model up, and it measures the result.

2. The vocabulary (a mini glossary)

These terms appear throughout. Skim them now; refer back as needed.

Token - A chunk of text the model reads and writes - roughly a word or word-piece. Models think in tokens, not letters.

Inference - Using a trained model to generate answers. (Training = building the model; inference = running it.)

GPU - Graphics Processing Unit. A chip with thousands of small cores that do the model's math in parallel. The project was tested on a free 'T4' GPU.

Latency - How long you wait. Two kinds matter: TTFT (time to first token - the initial pause before the first word) and TPOT (time per output token - the gap between each following word). Lower is better.

Throughput - How much work gets done per second, measured in tokens/second. Higher is better. Latency is about one user's wait; throughput is about serving many users.

Batch / batching - Handling several requests together in one go. Like a bus carrying many passengers per trip instead of one - far more efficient.

Quantization - Storing the model's numbers with less precision (e.g. 4-bit instead of 16-bit) to make it smaller and faster, with little quality loss. Like saving a photo as a smaller file that still looks fine.

Baseline - The plain, standard way of doing something, used as the reference to measure improvements against. Here the baseline is 'eager' PyTorch.

3. The big picture - what this project is

It is two things at once: an assembly line that takes a large AI model and makes it run faster on a GPU, and a measurement lab that times everything precisely so the speed-ups are proven, not merely claimed.

A good analogy: a workshop that takes a fast-but-thirsty car engine, tunes it for efficiency, and then puts it on a dynamometer to measure the exact horsepower gained. This project does that for an AI model - tune, then measure.

The work was split into six stages. Each stage improves or measures the model, and the output of one stage feeds the next.

4. How it works - the six stages

Stage 1 - Export

What: convert the model from its original format into a standard, portable one called ONNX. Why: optimization tools speak ONNX, not the original format. Analogy: translating a recipe into a universal format any kitchen can follow.

Stage 2 - Optimize

Two techniques make the model faster and smaller:

- TensorRT engine build: re-compile the model specifically for the exact GPU it will run on, so it squeezes the most out of that hardware. Like getting a tailored suit instead of one off the rack.
- Quantization: shrink the model by storing its numbers with less precision (4-bit). In this project a 2.2 GB model became about 0.76 GB - roughly 3x smaller - while still working correctly.

Stage 3 - Serve

One unified way to actually run the model, with three interchangeable 'engines' under the hood so they can be compared fairly:

- eager - the plain, standard way (the baseline to beat).
- ONNX - runs the portable exported model.
- vLLM - a state-of-the-art serving system. Its key trick is continuous batching: it keeps the GPU busy by constantly slotting in new requests, instead of idling between them. This is the fast one.

Stage 4 - Profile

Put a stopwatch and a power meter on the model: measure the time to the first word (TTFT), the speed of each following word (TPOT), the tokens per second, the GPU memory used, and the electricity drawn (in watts). This is how 'faster' becomes a number.

Stage 5 - Benchmark

Run the model under many different workloads - small vs large batches, short vs long inputs - and automatically record every result to a spreadsheet (CSV/JSON). This is the controlled experiment that produces the evidence.

Stage 6 - Analyze

Turn those spreadsheets into charts and a clear bottom line: how much faster did we actually get, and where?

5. What was achieved

Everything was built AND proven on a real GPU (a free Google Colab T4), end to end. Using TinyLlama (a small real model) with identical hardware across engines, the measured results were:

- vLLM delivered about 2.8x the throughput of the standard baseline on average (up to ~3.2x), peaking at ~1,159 tokens/second.
- vLLM cut the per-token wait by about 64% (roughly 12 ms vs 34 ms) - answers stream out noticeably more smoothly.
- Batching alone gave the baseline ~24x more throughput (28 -> 686 tokens/sec going from 1 request at a time to 32) - proof that keeping the GPU busy matters enormously.
- Quantization made the model ~3x smaller while still running correctly.
- Power and memory were captured live (~45-69 watts under load), so efficiency is quantified, not guessed.

What does '2.8x throughput' mean in practice? On the same GPU, the optimized setup serves nearly three times as many users per second - or the same users at roughly a third of the cost. The committed chart (docs/benchmarks/) shows vLLM pulling clearly ahead as the batch size grows.

These numbers come from a small model on modest free hardware. The same pipeline is built to run a much larger model (LLaMA 3 8B) on a powerful GPU, where the absolute gains are larger - only the hardware changes, not the code.

6. The hard part (and why it matters)

Building the pipeline was only half the work. The other half was making it run on real machines, which fought back constantly. These are the kinds of problems that do not appear in tutorials but dominate real engineering:

- Mismatched software versions. The newest tools for one stage demanded a different version of a shared library than the tools for another stage. The fix: give each stage its own clean workspace, the way a kitchen separates prep, cooking, and plating.
- GPU driver mismatches. Several libraries shipped builds for a newer GPU platform than the test machine had, causing cryptic 'file not found' crashes. Each was diagnosed and pinned to the matching version.
- A five-layer puzzle to get the fast (vLLM) engine running: a wrong package build, a missing system file, a library-loading-path problem, and a process-startup conflict - unwound one by one, then baked into the code so it self-heals and nobody repeats the ordeal.
- Crashes mid-experiment. The fast engine ran out of memory at very large batch sizes. Rather than lose the whole run, the benchmark was made crash-resilient: it saves every result it has gathered so far.

Every one of these was found by actually running the system, fixed, and locked in with automated tests so it stays fixed.

7. How quality was kept high

- Automated tests run on every change on an ordinary laptop (no GPU needed), catching mistakes early.
- Every GPU-only piece was validated on a real GPU before being marked done - nothing was assumed to

work.

- Every fix and decision is documented and version-controlled, so the project is reproducible and the reasoning is preserved.

These are the everyday habits of good engineering: test early, prove on real hardware, and write down why.

8. Why it matters and what's next

Faster, cheaper AI inference means lower running costs, snappier responses for users, and serving more people with the same hardware. This project delivers a complete, measured, reproducible path to that - and the evidence to back it up.

Natural next steps: run the full pipeline on a larger model (LLaMA 3 8B) on a datacenter GPU for bigger headline numbers; add deeper profiling (Nsight) on a dedicated machine; and explore advanced tricks like speculative decoding.

Tech stack: PyTorch, CUDA, TensorRT, ONNX, vLLM, NVML. Status: end-to-end pipeline complete and validated on GPU. Where to look in the repo: README.md (technical overview + results chart), CLAUDE.md (full phase log), src/ (the code), notebooks/ (the GPU runs), docs/benchmarks/ (the measured data + chart).